

Generating Letter Combinations of a Phone Number Using Recursion and Backtracking in Java

1. Basic Information

Experiment No:	_____
Date:	_____
Title:	Generating Letter Combinations of a Phone Number Using Recursion and Backtracking in Java
Language:	Java

2. Objective / Aim

To write a Java program that accepts a string of digits (2-9) and generates all possible letter combinations that the digits could represent on a traditional telephone keypad, using a recursive backtracking approach with a **HashMap** for digit-to-letter mapping.

3. System Requirements

3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 / AMD equivalent or higher
RAM	2 GB (4 GB recommended)
Hard Disk	500 MB free space
Input / Output	Keyboard, Monitor

3.2 Software Requirements

Component	Requirement
Operating System	Windows 10/11, Linux or macOS
Language / Compiler	Java Development Kit (JDK) 8 or higher
Editor / IDE	VS Code, IntelliJ IDEA, Eclipse, NetBeans or any text editor
Terminal	Command Prompt / PowerShell / Bash

4. Algorithm / Logic Description

The program treats the problem as building a string one character at a time. Each digit maps to a group of letters; at every digit we try each letter, move to the next digit (recursion) and, once all digits are consumed, store the formed string.

Step 1: Store the keypad mapping in a HashMap (2 → abc, 3 → def, ... 9 → wxyz).

Step 2: Read the digit string from the user.

Step 3: If the string is empty, print nothing; otherwise call `generate(digits, 0, "", result)`.

Step 4 (Base case): If `index == digits.length()`, add current to the result list and return.

Step 5 (Recursive case): Get the letters for `digits.charAt(index)`. For each letter, call `generate(digits, index+1, current + letter, result)`.

Step 6: After recursion finishes, print every string stored in the result list.

Trace for input 23:

Level (index)	Digit	Letters tried	Partial strings
0	2	a, b, c	a / b / c
1	3	d, e, f	ad, ae, af, bd, be, bf, cd, ce, cf
2 (base)	-	-	9 combinations stored

Complexity: Time $O(4^n \times n)$, Space $O(n)$ recursion depth (excluding output), where n is the number of digits and 4 is the maximum letters per digit (digits 7 and 9).

5. UML Class Diagram

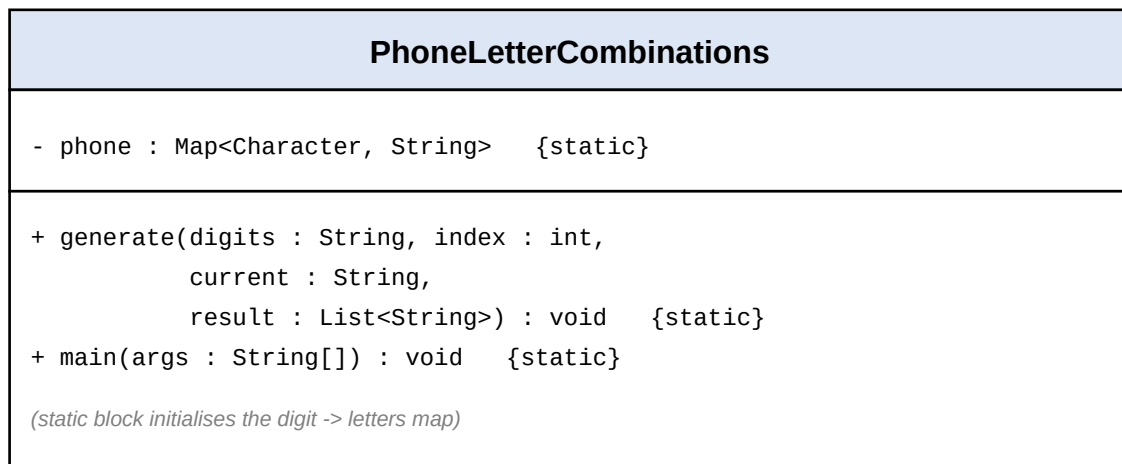


Figure 1: UML class diagram of PhoneLetterCombinations

6. Source Code

```
import java.util.*;

public class PhoneLetterCombinations {

    static Map<Character, String> phone = new HashMap<>();

    static {
        phone.put('2', "abc");
        phone.put('3', "def");
        phone.put('4', "ghi");
        phone.put('5', "jkl");
        phone.put('6', "mno");
        phone.put('7', "pqrs");
        phone.put('8', "tuv");
        phone.put('9', "wxyz");
    }

    static void generate(String digits, int index,
                        String current, List<String> result) {

        if (index == digits.length()) {
            result.add(current);
            return;
        }

        String letters = phone.get(digits.charAt(index));

        for (char ch : letters.toCharArray()) {
            generate(digits, index + 1,
                    current + ch, result);
        }
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter phone digits (2-9): ");
        String digits = sc.nextLine();
    }
}
```

```

        List<String> result = new ArrayList<>();

        if (digits.length() > 0) {
            generate(digits, 0, "", result);
        }

        System.out.println("Letter Combinations:");

        for (String combination : result) {
            System.out.println(combination);
        }

        sc.close();
    }
}

```

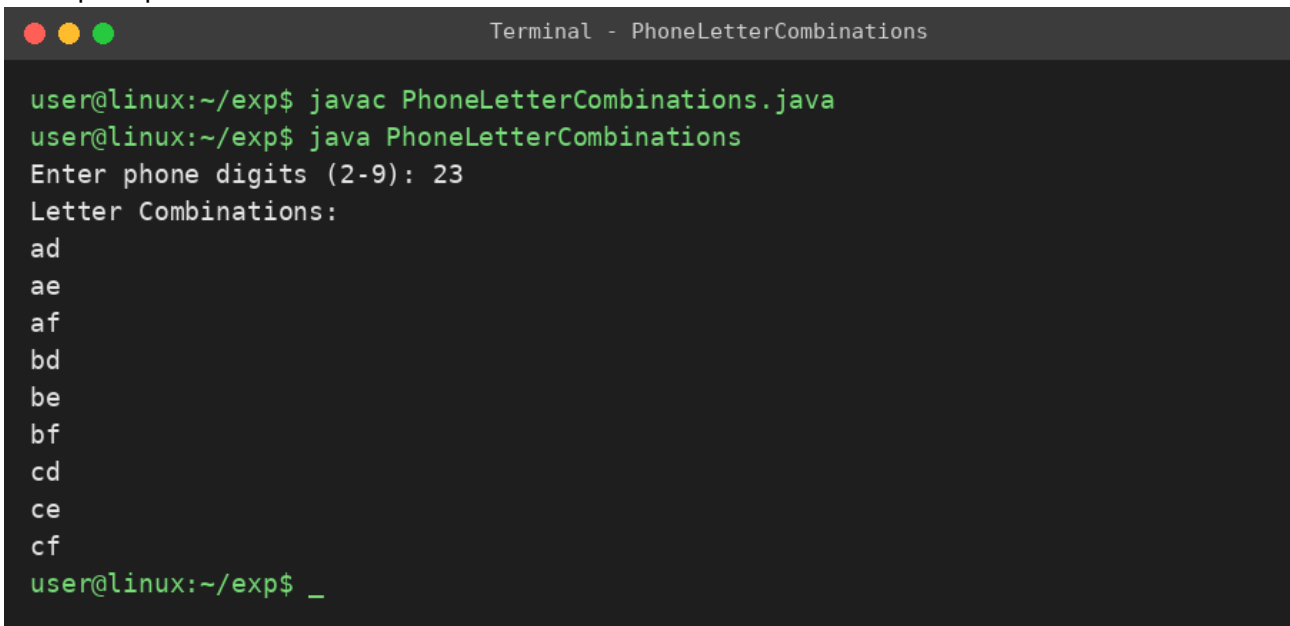
7. Compilation and Execution Commands

Purpose	Command
Check Java version	java -version
Compile	javac PhoneLetterCombinations.java
Run	java PhoneLetterCombinations

Save the code in a file named exactly `PhoneLetterCombinations.java` (file name must match the public class name), open a terminal in that folder and run the commands above in order.

8. Output

Sample input: 23

A terminal window titled "Terminal - PhoneLetterCombinations" is shown. The prompt is "user@linux:~/exp\$". The user enters "javac PhoneLetterCombinations.java" and then "java PhoneLetterCombinations". The program prompts "Enter phone digits (2-9): 23" and then "Letter Combinations:". It lists the following combinations: ad, ae, af, bd, be, bf, cd, ce, cf. The prompt returns to "user@linux:~/exp\$ _".

```
user@linux:~/exp$ javac PhoneLetterCombinations.java
user@linux:~/exp$ java PhoneLetterCombinations
Enter phone digits (2-9): 23
Letter Combinations:
ad
ae
af
bd
be
bf
cd
ce
cf
user@linux:~/exp$ _
```

Figure 2: Program output for input 23

9. Result / Conclusion

The program was successfully compiled and executed. It generated all 9 letter combinations for the input 23, demonstrating recursion and backtracking to enumerate combinations efficiently.